PART IIB REVISION NOTES

4F10: Deep Learning and Structured Data

Qianshuo (Harry) Ye
April 2026

1 Classifiers

Bayes's decision rule: for a classifier $f(\boldsymbol{x}^*, \boldsymbol{\theta})$, it generates a decision $\hat{y} \in \{\omega_1, \cdots, \omega_K\}$ by minimising the expected loss:

$$\hat{y} = \arg\min_{\omega} \left\{ \sum_{i=1}^{K} \mathcal{L}(\omega, \omega_i) P(\omega_i | \boldsymbol{x}^*) \right\}$$

Training a classifier: to train a classifier, minimise the expected loss

$$\mathcal{L}_{\text{act}} = \int \left[\sum_{i=1}^{K} \mathcal{L}(f(\boldsymbol{x}, \boldsymbol{\theta}), \omega_i) P(\omega_i | \boldsymbol{x}) \right] p(\boldsymbol{x}) d\boldsymbol{x}$$

which can be approximated as the empirical loss

$$\mathcal{L}_{\text{emp}} = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f(\boldsymbol{x}, \boldsymbol{\theta}), y_i)$$

where $\boldsymbol{x}_i \sim p(\boldsymbol{x})$ and $y_i \sim P(\omega | \boldsymbol{x_i})$ are (observation, class label) pairs in a supervised training data $\mathcal{D}$. In the limit $N \to \infty$, $\mathcal{L}_{\text{emp}} = \mathcal{L}_{\text{act}}$ but in practice $\mathcal{L}_{\text{emp}} \leq \mathcal{L}_{\text{act}}$.

To assess the performance on unseen data, split data into *training* and *evaluation data*.

Probability of error: For a binary classification problem, the probability of error is the probability that a sample from class ω_1 is misclassified as ω_2 plus the probability that a sample from class ω_2 is misclassified as ω_1:

$$P(\text{error}) = \int_{\Omega_2} p(\omega_1, \boldsymbol{x}) d\boldsymbol{x} + \int_{\Omega_1} p(\omega_2, \boldsymbol{x}) d\boldsymbol{x}$$

Generative vs Discriminative models: Generative models model the joint distribution of observations and classes $p(\boldsymbol{x}, \omega | \boldsymbol{\theta})$, and the posterior can be obtained from Bayes' rule:

$$P(\omega_i | \boldsymbol{x}^*, \boldsymbol{\theta}) = \frac{p(\boldsymbol{x}^*, \omega_i | \boldsymbol{\theta})}{\sum_{j=1}^{K} p(\boldsymbol{x}^*, \omega_j | \boldsymbol{\theta})} = \frac{P(\omega_i) p(\boldsymbol{x}^* | \omega_i, \boldsymbol{\theta})}{\sum_{j=1}^{K} P(\omega_j) p(\boldsymbol{x}^* | \omega_j, \boldsymbol{\theta})}$$

where $P(\omega_i)$ is the prior probability of class ω_i.

Discriminative model directly model the posterior $P(\omega | \boldsymbol{x}^*, \boldsymbol{\theta})$.

Discriminative model estimation: Find parameters $\boldsymbol{\theta}$ that maximise the likelihood of the labels given observations:

$$\hat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} \left\{ \sum_{i=1}^{N} \log P(y_i | \boldsymbol{x}_i, \boldsymbol{\theta}) \right\}$$

Generative model estimation: Find parameters $\boldsymbol{\theta}$ that maximise the likelihood of the observations given labels, where a separate model is trained for each class ω_i:

$$\hat{\boldsymbol{\theta}}_i = \arg\max_{\boldsymbol{\theta}} \left\{ \sum_{j:y_j=\omega_i} \log P(\boldsymbol{x}_j | \omega_i, \boldsymbol{\theta}) \right\}$$

To estimate the prior, simply count the number of samples in each class $P(\omega_i) = \frac{N_i}{N}$, or add 1 to all classes $P(\omega_i) = \frac{N_i+1}{N+K}$.

Multivariate Gaussian class conditional PDF: For a d-dimensional observation $\boldsymbol{x}$, the class conditional PDF is

$$p(\boldsymbol{x}|\omega_i, \boldsymbol{\theta}_i) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}_i|^{1/2}} \exp\left(-\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_i^{-1} (\boldsymbol{x} - \boldsymbol{\mu}_i) \right)$$

The maximum likelihood estimates of the mean and covariance are

$$\hat{\boldsymbol{\mu}}_i = \frac{1}{N_i} \sum_{j:y_j=\omega_i} \boldsymbol{x}_j$$
$$\hat{\boldsymbol{\Sigma}}_i = \frac{1}{N_i} \sum_{j:y_j=\omega_i} (\boldsymbol{x}_j - \hat{\boldsymbol{\mu}}_i)(\boldsymbol{x}_j - \hat{\boldsymbol{\mu}}_i)^T$$

Decision boundaries: The decision boundary is the set of points $\boldsymbol{x}$ where the posterior probabilities of two classes are equal

$$\log(P(\omega_1|\boldsymbol{x}, \boldsymbol{\theta})) = \log(P(\omega_2|\boldsymbol{x}, \boldsymbol{\theta}))$$

For a multivariate Gaussian class conditional PDF, the decision boundary is a quadratic function of $\boldsymbol{x}$:

$$\boldsymbol{x}^\top(\boldsymbol{\Sigma}_1^{-1} - \boldsymbol{\Sigma}_2^{-1})\boldsymbol{x} + 2(\boldsymbol{\Sigma}_2^{-1}\boldsymbol{\mu}_2 - \boldsymbol{\Sigma}_1^{-1}\boldsymbol{\mu}_1)^\top \boldsymbol{x} + \boldsymbol{\mu}_1^\top \boldsymbol{\Sigma}_1^{-1}\mu_1 - \boldsymbol{\mu}_2^\top \boldsymbol{\Sigma}_2^{-1}\boldsymbol{\mu}_2 + \log\left(\frac{|\boldsymbol{\Sigma}_1|}{|\boldsymbol{\Sigma}_2|}\right) - 2\log\left(\frac{P(\omega_1)}{P(\omega_2)}\right) = 0$$

The decision boundary is linear if the covariance matrices are the same $\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_2$.

2 Graphical Models

Independence: Two random variables X and Y are independent $(X \perp Y)$ if $P(X, Y) = P(X)P(Y)$.

Conditional independence: Two random variables X and Y are conditionally independent given a third variable Z $(X \perp Y|Z)$ if $P(X, Y|Z) = P(X|Z)P(Y|Z)$.

Conditional independence can be used to break down high-dimensional distributions into a product of simple factors evaluated only on a subset of variables.

Graphical models: factorisation based on conditional independence can be represented as a graph, where nodes represent random variables and edges represent dependencies between variables.

Bayesian networks: a directed acyclic graph where edges represent causal relationships between variables. Denoting PA_{X_i} as the set of parent nodes of X_i, the joint distribution can be factorised as $P(X_1, \ldots, X_d) = \prod_{i=1}^{d} P(X_i|\mathrm{PA}_{X_i})$. Denote non-descendants of X_i as ND_{X_i}, there exists conditional independencies $(X_i \perp \mathrm{ND}_{X_i}|\mathrm{PA}_{X_i})$.

Efficient marginalisation in graphical models: For a Bayesian network $A \to B \to C \to D$, the probability of d can be factorised as

$$p(d) = \sum_c \sum_b \sum_a p(d|c)p(c|b)p(b|a)p(a)$$

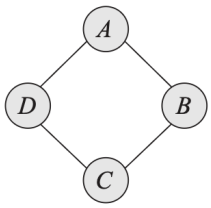
which have cost of $\mathcal{O}(n^4)$ when each variable takes n different values. Reording gives

$$p(d) = \sum_c p(d|c) \sum_b p(c|b) \sum_a p(b|a)p(a)$$

which have cost of $\mathcal{O}(n^2)$ (given by the largest factor).

Markov networks: an undirected graph where edges represent symmetric relationships between variables. The network is annotated with positive *potential* functions $\phi_1(\mathbf{D}_1), \ldots, \phi_k(\mathbf{D}_k)$, where $\{\mathbf{D}_i\}$ is the set of variables in the i-th *clique* – a fully connected set of nodes. The joint distribution can be factorised as $P(X_1, \ldots, X_d) = \frac{1}{Z} \prod_{i=1}^{k} \phi_i(\mathbf{D}_i)$, where Z is the partition function. The network encodes the conditional independencies $(A \perp B|C)$, where C separates A from B.

Graph:



Factorization:

$p(A, B, C, D) = \frac{1}{Z}\phi_1(A, B)\phi_2(B, C)\phi_3(C, D)\phi_4(A, D)$

Conditional Independencies:

$(A \perp C|B, D)$, $(B \perp D|A, C)$

3 Latent Variable and Sequence Models

3.1 Latent Variable Models

Gaussian mixture model: a model where each component is a Gaussian distribution:

$$p(\boldsymbol{x}) = \sum_{m=1}^{M} P(c_m)p(\boldsymbol{x}|c_m) = \sum_{m=1}^{M} P(c_m)\frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}_m|^{1/2}} \exp\left(-\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_m)^T \boldsymbol{\Sigma}_m^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_m)\right)$$

Factor analysis: a Bayesian network where the observed variable $\boldsymbol{x}$ is generated from a continuous latent variable $\boldsymbol{z}$ with lower dimension:

$$p(\boldsymbol{x}) = \int p(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})d\boldsymbol{z}$$

If $p(\boldsymbol{x}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ and $p(\boldsymbol{z}) = \mathcal{N}(C\boldsymbol{z}, \boldsymbol{\Sigma}_{\text{diag}})$, $p(\boldsymbol{x}) = \mathcal{N}(\mathbf{0}, CC^\top + \boldsymbol{\Sigma}_{\text{diag}})$.

Auxiliary function: For Gaussian mixture model, a change of parameters from $\boldsymbol{\theta}^{(k)}$ to $\boldsymbol{\theta}^{(k+1)}$ gives a change of log-likelihood

$$\begin{aligned}
\log p(\boldsymbol{x}|\boldsymbol{\theta}^{(k+1)}) - \log p(\boldsymbol{x}|\boldsymbol{\theta}^{(k)}) &= \sum_{i=1}^{N} \log \frac{p(\boldsymbol{x}_i|\boldsymbol{\theta}^{(k+1)})}{p(\boldsymbol{x}_i|\boldsymbol{\theta}^{(k)})} \\
&= \sum_{i=1}^{N} \log \left(\sum_{m=1}^{M} \frac{P(c_m|\boldsymbol{x}_i, \boldsymbol{\theta}^{(k)})p(\boldsymbol{x}_i, c_m|\boldsymbol{\theta}^{(k+1)})}{P(c_m|\boldsymbol{x}_i, \boldsymbol{\theta}^{(k)})p(\boldsymbol{x}_i|\boldsymbol{\theta}^{(k)})} \right) \\
\text{(Jensen's inequality)} \quad &\geq \sum_{i=1}^{N} \sum_{m=1}^{M} P(c_m|\boldsymbol{x}_i, \boldsymbol{\theta}^{(k)}) \log \left(\frac{p(\boldsymbol{x}_i, c_m|\boldsymbol{\theta}^{(k+1)})}{p(\boldsymbol{x}_i, c_m|\boldsymbol{\theta}^{(k)})} \right) \\
&= \mathcal{Q}(\boldsymbol{\theta}^{(k)}, \boldsymbol{\theta}^{(k+1)}) - \mathcal{Q}(\boldsymbol{\theta}^{(k)}, \boldsymbol{\theta}^{(k)})
\end{aligned}$$

where $\mathcal{Q}(\boldsymbol{\theta}^{(k)}, \boldsymbol{\theta}^{(k+1)}) = \sum_{i=1}^{N} \sum_{m=1}^{M} P(c_m|\boldsymbol{x}_i, \boldsymbol{\theta}^{(k)}) \log p(\boldsymbol{x}_i, c_m|\boldsymbol{\theta}^{(k+1)})$ is known as the *auxiliary function*. For factor analysis, $\mathcal{Q}(\boldsymbol{\theta}^{(k)}, \boldsymbol{\theta}^{(k+1)}) = \sum_{i=1}^{N} \int p(\boldsymbol{z}|\boldsymbol{x}_i, \boldsymbol{\theta}^{(k)}) \log p(\boldsymbol{x}_i, \boldsymbol{z}|\boldsymbol{\theta}^{(k+1)})d\boldsymbol{z}$.

Expectation maximisation: Given a distribution $q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}})$, the log-likelihood can be decomposed as

$$\begin{aligned}
\mathcal{L}(\theta) = \log p(\boldsymbol{x}|\boldsymbol{\theta}) &= \int q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}}) \log p(\boldsymbol{x}|\boldsymbol{\theta}) d\boldsymbol{z} \\
&= \int q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}}) \log \frac{p(\boldsymbol{x}, \boldsymbol{z}|\boldsymbol{\theta})}{p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{\theta})} d\boldsymbol{z} \\
&= \mathbb{E}_{q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}})} \left[\frac{p(\boldsymbol{x}, \boldsymbol{z}|\boldsymbol{\theta})}{p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{\theta})} \right] \\
&= \mathbb{E}_{q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}})} \left[\frac{p(\boldsymbol{x}, \boldsymbol{z}|\boldsymbol{\theta})}{q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}})} \right] + \underbrace{\int q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}}) \log \frac{q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}})}{p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{\theta})} d\boldsymbol{z}}_{\text{KL divergence, non-negative}} \\
&\geq \mathbb{E}_{q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}})} \left[\frac{p(\boldsymbol{x}, \boldsymbol{z}|\boldsymbol{\theta})}{q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}})} \right] = \mathcal{F}(q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}}), \boldsymbol{\theta})
\end{aligned}$$

The EM algorithm iteratively maximises the lower bound $\mathcal{F}(q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}}), \boldsymbol{\theta})$ by alternating between E-step and M-step:

- E-step: $q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}}^{(k)}) = p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{\theta}^{(k)})$ (might be intractable, where decrease of $\mathcal{L}(\boldsymbol{\theta})$ is not guaranteed)
- M-step: $\boldsymbol{\theta}^{(k+1)} = \arg\max_{\boldsymbol{\theta}} \mathcal{F}(q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}}^{(k)}), \boldsymbol{\theta})$

which yields the sequence of inequalities for iteration $k+1$:

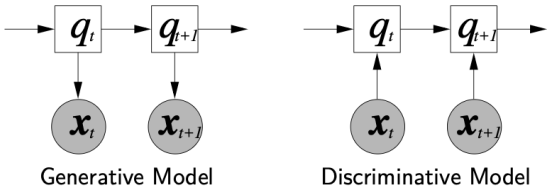
$$\mathcal{L}(\boldsymbol{\theta}^{(k+1)}) \geq \mathcal{F}(q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}}^{(k+1)}), \boldsymbol{\theta}^{(k+1)}) \geq \mathcal{F}(q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}}^{(k)}), \boldsymbol{\theta}^{(k)}) = \mathcal{L}(\boldsymbol{\theta}^{(k)})$$

Mean-field approximation: For a distribution $q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}})$, the mean-field approximation factorises it as $q(\boldsymbol{z}, \tilde{\boldsymbol{\theta}}) = \prod_i q_i(\boldsymbol{z}_i, \tilde{\boldsymbol{\theta}}_i)$.

3.2 Sequence Models

Hidden Markov model: a *generative* model that models the joint probability of observations and latent variables. Given a sequence of observations $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T$, add discrete latent variables q_t. Assuming conditional independence (Markov property) $P(q_t|q_{t-1}, \ldots, q_0) = P(q_t|q_{t-1})$ and $p(\boldsymbol{x}_t|q_t, \boldsymbol{x}_{t-1}, \ldots, \boldsymbol{x}_1) = p(\boldsymbol{x}_t|q_t)$, the likelihood of observations is

$$p(\boldsymbol{x}_0, \ldots, \boldsymbol{x}_T) = \sum_{\boldsymbol{q} \in \boldsymbol{Q}_T} P(q_0) \prod_{t=1}^{T} P(q_t|q_{t-1}) P(\boldsymbol{x}_t|q_t)$$



Viterbi approximation: approximate the likelihood of observations by the most likely sequence of latent variables:

$$p(\boldsymbol{x}_0, \ldots, \boldsymbol{x}_T) \approx \max_{\boldsymbol{q} \in \boldsymbol{Q}_T} p(\boldsymbol{x}_0, \ldots, \boldsymbol{x}_T, \boldsymbol{q})$$

Viterbi algorithm: a dynamic programming algorithm to find the most likely sequence of latent variables $\hat{\boldsymbol{q}} = \arg\max_{\boldsymbol{q} \in \boldsymbol{Q}_T} p(\boldsymbol{x}_0, \ldots, \boldsymbol{x}_T, \boldsymbol{q})$.

- Initialisation: (`LZERO`$=\log(0)$)

$$\phi_1(0) = 0.0, \quad \phi_j(0) = \texttt{LZERO}, 1 < j < N, \quad \phi_1(t) = \texttt{LZERO}, 1 \leq t \leq T$$

- Recursion:

  `for` $t = 1, \ldots, T$

  `for` $j = 2, \ldots, N-1$

$$\phi_j(t) = \max_{1 \leq k < N} \{\phi_k(t-1) + \log(a_{kj})\} + \log(b_j(\boldsymbol{x}_t))$$

- Termination:

$$\log(p(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T, \hat{\boldsymbol{q}})) = \max_{1 < k < N} \{\phi_k(T) + \log(a_{kN})\}$$

Forward-backward algorithm: Define forward probability term $\alpha_j(t) = \log p(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_t, q_t = s_j)$ and backward probability term $\beta_j(t) = \log p(\boldsymbol{x}_{t+1}, \ldots, \boldsymbol{x}_T | q_t = s_j)$, and the posterior probability of latent variable q_t is

$$P(q_t = s_j | \boldsymbol{x}_1, \ldots, \boldsymbol{x}_T) = \frac{\exp(\alpha_j(t) + \beta_j(t))}{\sum_{k=1}^{M} \exp(\alpha_k(t) + \beta_k(t))}$$

Maximum entropy Markov model: a *discriminative* model that directly models the posterior probability of latent variables given observations:

$$P(q_0, \ldots, q_T | \boldsymbol{x}_1, \ldots, \boldsymbol{x}_T) = \frac{1}{Z} \exp\left(\sum_{i=1}^{D} \lambda_i f_i(q_0, \ldots, q_T, \boldsymbol{x}_1, \ldots, \boldsymbol{x}_T) \right)$$

Linear chain conditional random field (CRF): a maximum entropy Markov model that assume conditional independences similar to HMM, where the posterior probability is

$$P(q_0, \ldots, q_T | \boldsymbol{x}_1, \ldots, \boldsymbol{x}_T) = \frac{1}{Z} \exp\left(\sum_{t=1}^{T} \left(\sum_{i=1}^{D_t} \lambda_i^t f_i(q_{t-1}, q_t) + \sum_{i=1}^{D_a} \lambda_i^a f_i(q_t, \boldsymbol{x}_t) \right) \right)$$

where D_t is the number of transition style features and D_a is the number of observation (acoustic) style features. Extending condtional independeces to the previous state gives $P(q_0, \ldots, q_T | \boldsymbol{x}_1, \ldots, \boldsymbol{x}_T) = \frac{1}{Z} \exp\left(\sum_{t=1}^{T} \sum_{i=1}^{D} \lambda_i f_i(q_t, q_{t-1}, \boldsymbol{x}_t) \right)$.

General sequence CRF: incorporate cliques between variables, where the posterior probability is

$$p(q_0, \ldots, q_T | \boldsymbol{x}_1, \ldots, \boldsymbol{x}_T) = \frac{1}{Z} \exp\left(\sum_{t=1}^{T} \sum_{c \in \boldsymbol{C}} \boldsymbol{\lambda}_c^\top \boldsymbol{f}(\boldsymbol{q}_{c_t}, \boldsymbol{x}_1, \ldots, \boldsymbol{x}_T, t) \right)$$

To train a CRF, maximise the posterior of $\boldsymbol{q}_{1:T}$ given $\boldsymbol{x}_{1:T}$.

4 Deep Learning

4.1 Neural Networks

Universal approximation theorem: with enough hidden units, a shallow neural nework can describe any continuous functino on a compact subset of $\mathbb{R}^D$ to arbitrary precision.

Number of regions created by output planes: The number of regions created by $D > D_i$ planes in D_i dimensions is $\sum_{i=0}^{D_i} \binom{D}{i}$.

Hyperparameters of a neural network: number of layers = depth of network, number of hidden units per layer = width of network.

4.2 Training Neural Networks

Dimensionality of derivatives: Any function $f(x) \to y$ where $x \in \mathbb{R}^{M \times N \times P}, y \in \mathbb{R}^{R \times S \times T}$ has a Jacobian $f'(x) \to J$ where $J \in \mathbb{R}^{R \times S \times T \times M \times N \times P}$. The large intermediates Jacobians can be avoided by reverse derivative using Vector-Jacobian Products.

Vector-Jacobian Product: Defined as

$$\vec{f}(x; v) = v \cdot f'(x)$$

Basic rule: every line $p, q = f(x, y)$ in the forward pass becomes $x', y' = \vec{f}(x, y; p', q')$.

The multiply-and-add function $\mu(A, x, b) = Ax+b$ have VJP $\vec{\mu}(A, x, b; v) = (vx^\top, A^\top v, v)$.

Stochastic gradient descent: use a mini-batch of data to compute the gradient and update parameters to introduce noise to escape local minima and saddle points:

$$\phi_{t+1} \leftarrow \phi_t - \alpha \sum_{i \in \mathcal{B}_t} \frac{\partial l_i[\phi_t]}{\partial \phi_t}$$

The mini-batches are sampled from the dataset without replacement, and one pass through the data is an *epoch*.

Momentum: weighted sum of current gradient and previous gradient

$$\begin{aligned} m_{t+1} &\leftarrow \beta\, m_t + (1-\beta) \sum_{i \in \mathcal{B}_t} \frac{\partial l_i[\phi_t]}{\partial \phi_t} \\ \phi_{t+1} &\leftarrow \phi_t - \alpha\, m_{t+1} \end{aligned}$$

Adaptive moment estimation (Adam): measure mean and pointwise squared gradient with momentum

$$\begin{aligned} m_{t+1} &\leftarrow \beta\, m_t + (1-\beta) \frac{\partial L[\phi_t]}{\partial \phi_t} \\ v_{t+1} &\leftarrow \gamma\, v_t + (1-\gamma) \left(\frac{\partial L[\phi_t]}{\partial \phi_t} \right)^2 \\ \tilde{m}_{t+1} &\leftarrow \frac{m_{t+1}}{1-\beta^{t+1}} \\ \tilde{v}_{t+1} &\leftarrow \frac{v_{t+1}}{1-\gamma^{t+1}} \\ \phi_{t+1} &\leftarrow \phi_t - \alpha \frac{\tilde{m}_{t+1}}{\sqrt{\tilde{v}_{t+1}} + \epsilon} \end{aligned}$$

He initialisation: The variance of $y = \phi(Ax)$, where ϕ is the ReLU and $A \in \mathbb{R}^{M \times N}$ is rougly

$$\sigma_y^2 \leq \frac{N}{2} \sigma_A^2 \sigma_x^2$$

So initialise A with variance $\sigma_A^2 = \frac{2}{N}$ to prevent vanishing or exploding gradients.

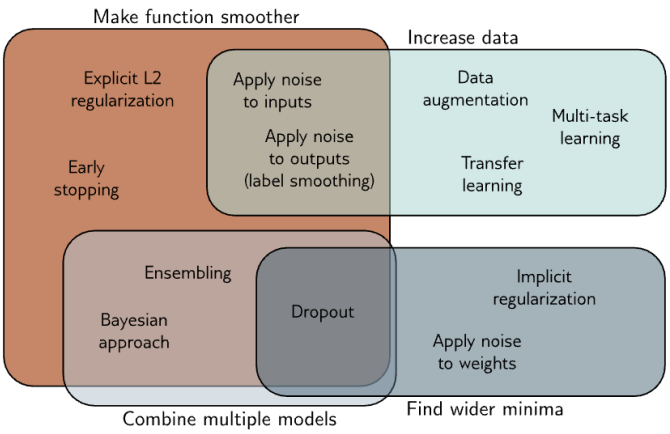
Stable log-sum-exp: $\text{logsumexp}(\boldsymbol{y}) = \text{logsumexp}(\boldsymbol{y} - \max(\boldsymbol{y})) + \max(\boldsymbol{y})$

Newtons's method: update parameters by the Hessian matrix H and the gradient g:

$$\theta_{t+1} \leftarrow \theta_t - \frac{1}{2} H^{-1} g$$

Damped Newton's method adds a damping term λ: $\theta_{t+1} \leftarrow \theta_t - \frac{1}{2}(H + \lambda I)^{-1} g$. In Adam, $m_i \approx \frac{g_i}{\sqrt{g_i}}$ and $H \approx \text{diag}(g_i)^{\frac{1}{2}}$.

Regularisation: methods to reduce the generalisation gap due to overfitting to training data or model being uncontrained in areas with no training examples.



4.3 Architectures

Layer norm: normalise a vector $z \in \mathbb{R}^D$ as

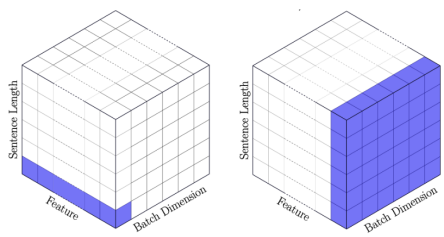
$$N(z)_i = \frac{z_i - \text{mean}(z)}{\text{std}(z) + \epsilon}$$

where mean $= \frac{1}{D}\sum_{i=1}^{D} z_i$ and std $= \sqrt{\frac{1}{D}\sum_{i=1}^{D}(z_i - \text{mean}(z))^2}$.

Batch norm: normalise a batch of vectors $\boldsymbol{z} \in \mathbb{R}^{B \times D}$ as

$$N(z)_{i,j} = \frac{z_{i,j} - \text{mean}(z)_j}{\text{std}(z)_j + \epsilon}$$

where $\text{mean}(z)_j = \frac{1}{B}\sum_{i=1}^{B} z_{i,j}$ and $\text{std}(z)_j = \sqrt{\frac{1}{B}\sum_{i=1}^{B}(z_{i,j} - \text{mean}(z)_j)^2}$.



Translation invariance: a function F is invariant if for all transformations ϕ, $F(X * \phi) = F(X)$.

General assumptions on transformations: 1) there is always an inverse ϕ^{-1}; 2) a composition of two transformations can be represented as a single transformation $\phi_1 * \phi_2$.

Convolution kernel: a kernel applied across all positions to weigh neighbouring inputs for each output – translation invariance, with parameters:

- *Stride:* shift by k positions for each output. Larger stride decreases size of output relative to input.
- *Kernel size:* the number of inputs for each output. Larger kernel size combines information from a larger area.

- *Dilated convolution:* intersperse kernel values with zeros. Larger dilation combines information from a larger area with fewer parameters.
- *Channel:* apply several convolution kernels to the same input to produce several outputs (feature maps). Larger number of channels allows the model to learn more features.

Downsampling: introduces an information bottleneck. Methods include: strided convolution, max pooling, mean pooling.

Upsampling: expand feature maps to a larger size. Methods include: duplicate, max upsampling, bilinear interpolation.

Residual connection: add the input to the output of a layer to avoid vanishing or exploding gradients and smooths the loss landscape.

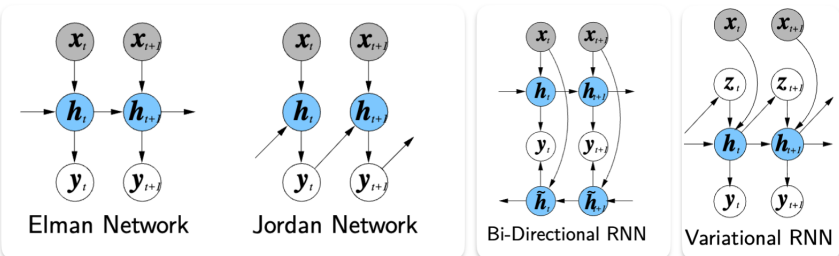
Lottery ticket hypothesis: a small subnetwork within a larger network can be trained in isolation to achieve comparable performance to the original network.

5 Deep Learning for Sequences

5.1 Sequence I/O Pair Modelling

Recurrent neural network (RNN): network with a history vector h_t used to encode causal memory

$$\begin{aligned} \boldsymbol{h}_t &= f^h(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t) \text{ (Elman network)} \\ \text{or } \boldsymbol{h}_t &= f^h(\boldsymbol{y}_{t-1}, \boldsymbol{x}_t) \text{ (Jordan network)} \\ \boldsymbol{y}(\boldsymbol{x}_{1:t}) &= f^y(\boldsymbol{h}_t) \end{aligned}$$

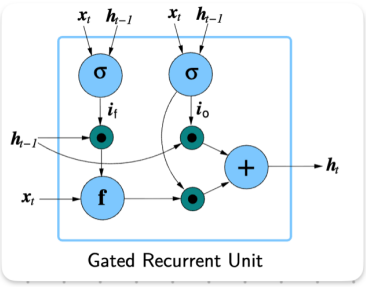


Variational RNN introduce a latent variable sequence $\boldsymbol{z}_{1:T}$

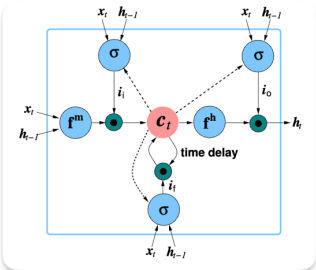
$$p(\boldsymbol{y}_t|\boldsymbol{x}_{1:t}) \approx \int p(\boldsymbol{y}_t|\underbrace{\boldsymbol{h}_{t-1}, \boldsymbol{z}_t, \boldsymbol{x}_t}_{\text{determines } \boldsymbol{h}_t})p(\boldsymbol{z}_t|\boldsymbol{h}_{t-1})d\boldsymbol{z}_t$$

Gated recurrent unit (GRU): using forget gate i_f and output gate i_o to control the contribution of last history $\boldsymbol{h}_{t-1}$ and current input $\boldsymbol{x}_t$ to the current history vector $\boldsymbol{h}_t$.

Long short-term memory (LSTM): using forget gate i_f, input gate i_i and output gate i_o to update memory cell $\boldsymbol{c}_t$ ('long-term memory') and history vector $\boldsymbol{h}_t$ ('short-term



$$
\begin{aligned}
\boldsymbol{i}_{\mathtt{f}} &= \boldsymbol{\sigma}(\mathbf{W}^{\mathtt{f}}_{\mathtt{f}}\boldsymbol{x}_t + \mathbf{W}^{\mathtt{r}}_{\mathtt{f}}\boldsymbol{h}_{t-1} + \boldsymbol{b}_{\mathtt{f}}) \\
\boldsymbol{i}_{\mathtt{o}} &= \boldsymbol{\sigma}(\mathbf{W}^{\mathtt{f}}_{\mathtt{o}}\boldsymbol{x}_t + \mathbf{W}^{\mathtt{r}}_{\mathtt{o}}\boldsymbol{h}_{t-1} + \boldsymbol{b}_{\mathtt{o}}) \\
\tilde{\boldsymbol{h}}_t &= \mathbf{f}(\mathbf{W}^{\mathtt{f}}_{\mathtt{h}}\boldsymbol{x}_t + \mathbf{W}^{\mathtt{r}}_{\mathtt{h}}(\boldsymbol{i}_{\mathtt{f}} \odot \boldsymbol{h}_{t-1}) + \boldsymbol{b}_{\mathtt{h}}) \\
\boldsymbol{h}_t &= \boldsymbol{i}_{\mathtt{o}} \odot \boldsymbol{h}_{t-1} + (\mathbf{1} - \boldsymbol{i}_{\mathtt{o}}) \odot \tilde{\boldsymbol{h}}_t
\end{aligned}
$$



Forget gate ($\boldsymbol{i}_{\mathtt{f}}$), Input gate ($\boldsymbol{i}_{\mathtt{i}}$), Output gate ($\boldsymbol{i}_{\mathtt{o}}$)

$$
\begin{aligned}
\boldsymbol{i}_{\mathtt{f}} &= \boldsymbol{\sigma}(\mathbf{W}^{\mathtt{f}}_{\mathtt{f}}\boldsymbol{x}_t + \mathbf{W}^{\mathtt{r}}_{\mathtt{f}}\boldsymbol{h}_{t-1} + \mathbf{W}^{\mathtt{m}}_{\mathtt{f}}\boldsymbol{c}_{t-1} + \boldsymbol{b}_{\mathtt{f}}) \\
\boldsymbol{i}_{\mathtt{i}} &= \boldsymbol{\sigma}(\mathbf{W}^{\mathtt{f}}_{\mathtt{i}}\boldsymbol{x}_t + \mathbf{W}^{\mathtt{r}}_{\mathtt{i}}\boldsymbol{h}_{t-1} + \mathbf{W}^{\mathtt{m}}_{\mathtt{i}}\boldsymbol{c}_{t-1} + \boldsymbol{b}_{\mathtt{i}}) \\
\boldsymbol{i}_{\mathtt{o}} &= \boldsymbol{\sigma}(\mathbf{W}^{\mathtt{f}}_{\mathtt{o}}\boldsymbol{x}_t + \mathbf{W}^{\mathtt{r}}_{\mathtt{o}}\boldsymbol{h}_{t-1} + \mathbf{W}^{\mathtt{m}}_{\mathtt{o}}\boldsymbol{c}_t + \boldsymbol{b}_{\mathtt{o}})
\end{aligned}
$$

Memory Cell, history vector and gates are related by

$$
\begin{aligned}
\boldsymbol{c}_t &= \boldsymbol{i}_{\mathtt{f}} \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_{\mathtt{i}} \odot \mathbf{f}^{\mathtt{m}}(\mathbf{W}^{\mathtt{f}}_{\mathtt{c}}\boldsymbol{x}_t + \mathbf{W}^{\mathtt{r}}_{\mathtt{c}}\boldsymbol{h}_{t-1} + \boldsymbol{b}_{\mathtt{c}}) \\
\boldsymbol{h}_t &= \boldsymbol{i}_{\mathtt{o}} \odot \mathbf{f}^{\mathtt{h}}(\boldsymbol{c}_t)
\end{aligned}
$$

memory'). The memory cell weight matrices W_f^m, W_i^m, W_o^m are diagonal such that individual cell elements can be analysed explicitly.

Highway connections: gate input and output of a layer (rather than direct addition in residual connection).

5.2 Sequence to Target

Averaging: compute the average of states $\boldsymbol{h}_1, \ldots, \boldsymbol{h}_T$, where $\boldsymbol{h}_t = \mathcal{F}(\boldsymbol{x}_t)$.

RNN encoding: use the last state $\boldsymbol{h}_T$, where $\boldsymbol{h}_t = \mathcal{F}(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t)$.

Attention mechanism: compute a weighted average of states $\boldsymbol{h}_1, \ldots, \boldsymbol{h}_T$ to produce the target $\boldsymbol{c}$ given a query $\boldsymbol{q}$:

$$\boldsymbol{c} = \sum_{t=1}^{T} \alpha_t \boldsymbol{h}_t, \text{ where } \alpha_t = \frac{\exp(\mathcal{F}(\boldsymbol{h}_t, \tilde{\boldsymbol{h}}))}{\sum_{i=1}^{T} \exp(\mathcal{F}(\boldsymbol{h}_i, \tilde{\boldsymbol{h}}))}, \ \tilde{\boldsymbol{h}} = \mathcal{F}_q(\boldsymbol{q})$$

Standard forms of attention:

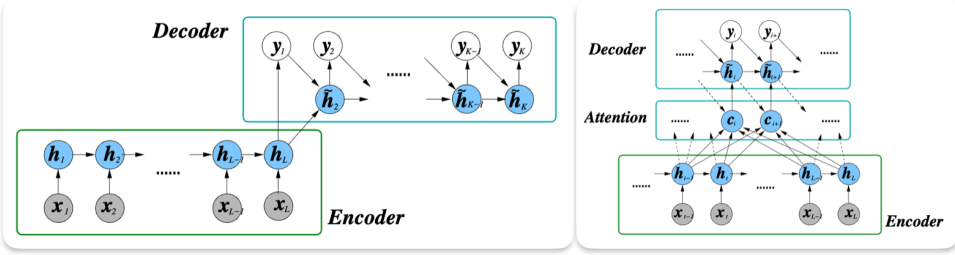
- *Dot-product attention:* $\mathcal{F}(\boldsymbol{h}_t, \tilde{\boldsymbol{h}}) = \boldsymbol{h}_t^\top W_{xq} \tilde{\boldsymbol{h}}$
- *Additive attention:* $\mathcal{F}(\boldsymbol{h}_t, \tilde{\boldsymbol{h}}) = \mathbf{w}^\top \tanh(W_x \boldsymbol{h}_t + W_q \tilde{\boldsymbol{h}})$

5.3 Sequence to Sequence

Encoder-decoder architecture: use an encoder to encode the input sequence into a fixed-length vector(s). and use a decoder to decode the vector into an output sequence.

$$p(\boldsymbol{y}_{1:K}|\boldsymbol{x}_{1:L}) = \prod_{i=1}^{K} p(y_i|\boldsymbol{y}_{1:i-1}, \boldsymbol{x}_{1:L}) \approx \prod_{i=1}^{K} p(y_i|\boldsymbol{y}_{i-1}, \tilde{\boldsymbol{h}}_{i-1}, \boldsymbol{c}), \text{ where } \boldsymbol{c} = \phi(\boldsymbol{x}_{1:L})$$

For RNN, $\boldsymbol{c} = \boldsymbol{h}_L$. For attention, the encoder produces a sequence of vectors $\boldsymbol{c}_i = \sum_{\tau=1}^{L} \alpha_{i\tau} \boldsymbol{h}_\tau$, where $\alpha_{i\tau} = \frac{\exp(\mathcal{F}(\boldsymbol{h}_\tau, \tilde{\boldsymbol{h}}_{i-1}))}{\sum_{j=1}^{L} \exp(\mathcal{F}(\boldsymbol{h}_j, \tilde{\boldsymbol{h}}_{i-1}))}$ (the query for i-th output is $\boldsymbol{y}_{1:i-1}$).



RNN Attention-based

Greedy decoding: at each step, choose the most likely output $\hat{y}_i = \arg\max_{y_i} p(y_i|\hat{\boldsymbol{y}}_{1:i-1}, \boldsymbol{x}_{1:L})$.

Word embeddings: represent words as vectors

$$\underbrace{w_i}_{\text{word}} \rightarrow \underbrace{\boldsymbol{t}_i}_{\text{one-hot encoding}} \rightarrow \underbrace{W\boldsymbol{t}_i = \boldsymbol{x}_i}_{\text{embedding}}$$

Word2Vec: learn word projection W by training a model to predict a word w_i or the word given its context (continuous bag of words) via maximising

$$\prod_{i=1}^{L} p(w_i|w_{i-k}, \ldots, w_{i-1}, w_{i+1}, \ldots, w_{i+k}) = \prod_{i=1}^{L} p(w_i|W(\boldsymbol{t}_{i-k}, \ldots, \boldsymbol{t}_{i-1}, \boldsymbol{t}_{i+1}, \ldots, \boldsymbol{t}_{i+k}))$$

Transformer: use multi-head attention for relationships within or between sequences

- *Scaled dot-product multi-head attention:*

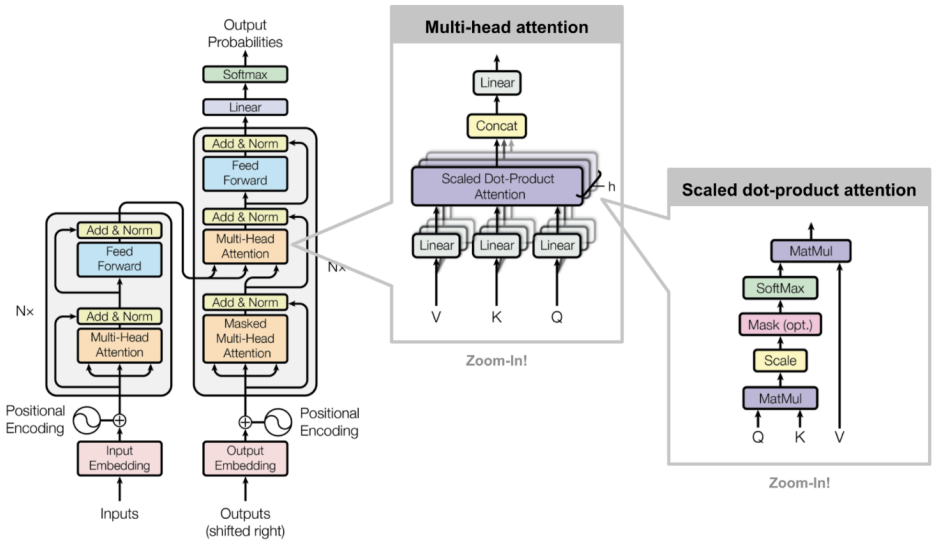
$$\alpha_{\tau i}^{(j)} = \frac{1}{Z} \exp\left(\frac{\overbrace{(W_q^{(j)}\boldsymbol{x}_\tau)^\top}^{\text{query}} \overbrace{(W_k^{(j)}\boldsymbol{x}_i)}^{\text{key}}}{\sqrt{d_k^{(j)}}} \right)$$

$$\boldsymbol{c}_i^{(j)} = \sum_{\tau=1}^{L} \alpha_{\tau i}^{(j)} \overbrace{W_v^{(j)}\boldsymbol{x}_\tau}^{\text{value}}$$

$$\boldsymbol{c} = \left[\boldsymbol{c}_i^{(1)\top}, \ldots, \boldsymbol{c}_i^{(H)\top}\right]^\top$$

- *Masked multi-head attention:* mask the attention weights $\alpha_{\tau i}^{(j)}$ for $\tau > i$ to prevent the model from attending to future inputs.
- *Multi-head cross attention:* use values and key from the encoder and query from the (masked) decoder.
- *Feedforward network:* apply to the output of multi-head attention.
- *Add and norm:* add the input to the output of each layer and apply layer norm.
- *Output layer:* use a linear projection to map the output of the decoder to the vocabulary size, and apply softmax to get an posterior distribution of next word.

BERT: predict one or more masked words in a sequence rather than the next word.



6 Ensemble Methods

Binary ensemble classifier: select the class with the most votes from a set of binary classifiers (majority voting). The probability of error is

$$P(\text{error}) = \sum_{i=\lceil \frac{M}{2} \rceil}^{M} \binom{M}{i} p^i (1-p)^{M-i}$$

Bayesian ensemble:

$$\hat{\omega} = \arg\max_{\omega} \left\{ \sum_{\mathcal{M}} \int P(\omega|\boldsymbol{x}^*, \mathcal{M}, \boldsymbol{\theta}) p(\mathcal{M}, \boldsymbol{\theta}|\mathcal{D}) d\boldsymbol{\theta} \right\}$$

where $\mathcal{M}$ is a model topology and $\boldsymbol{\theta}$ is the parameters of the model. The Monte-Carlo approximation is

$$\hat{\omega} = \arg\max_{\omega} \left\{ \frac{1}{M} \sum_{m=1}^{M} P(\omega|\boldsymbol{x}^*, \boldsymbol{\theta}^{(m)}) \right\}, \text{ where } \boldsymbol{\theta}^{(m)} \sim p(\mathcal{M}, \boldsymbol{\theta}|\mathcal{D})$$

Monte-Carlo dropout: generate ensemble by randomly de-activating nodes during training.

Random network initialisation: given a topology $\mathcal{M}$, initialise multiple network parameters as $\tilde{\boldsymbol{\theta}}^{(m)} \sim p(\boldsymbol{\theta}|\mathcal{M})$. Train each network using the same data.

Posterior combination: a simple way to combine the predictions of multiple models is to take the average of their posterior probabilities:

$$P(\omega|\boldsymbol{x}^*) = \frac{1}{M} \sum_{m=1}^{M} P(\omega|\boldsymbol{x}^*, \boldsymbol{\theta}^{(m)})$$

Teacher-student training: train a student model to mimic the predictions of a teacher model (rather than the true labels) by minimising the cross-entropy loss

$$\mathcal{F}_{ts} = -\sum_{i=1}^{N} \sum_{\omega} P(\omega|\boldsymbol{x}_i, \boldsymbol{\theta}_t) \log P(\omega|\boldsymbol{x}_i, \boldsymbol{\theta}_s)$$

The single teacher model can be replaced by an ensemble of models.

Mixture of experts: use a gating function $P(e_m|\boldsymbol{x}^*, \boldsymbol{\theta})$ to combine the predictions of multiple models

$$P(\omega|\boldsymbol{x}^*) = \sum_{m=1}^{M} P(e_m|\boldsymbol{x}^*, \boldsymbol{\theta}) P(\omega|\boldsymbol{x}^*, \boldsymbol{\theta}, e_m)$$

where each expert focus on different aspects of the data.

Switch transformer: use a router (equivalent to the gating function) to select between different feedforward networks.

7 Support Vector Machines

Maximum margin classifier: when the data is linearly separable, find the plane that maximises the margin – the distance to the closest point in each class. The data points on the margin are called *support vectors*. The magnitude of the margin is

$$\frac{w^\top (x_+ - x_-)}{2\|w\|} = \frac{1}{\|w\|}$$

where x_+ and x_- are the support vectors of the positive and negative class respectively.

Optimisation with inequality constraints: For the constrained optimisation problem: maximise $f(x, y)$ subject to $g(x, y) \geq 0$, the Lagragian function is

$$\mathcal{L}(x, y, \lambda) = f(x, y) + \lambda g(x, y)$$

The solution must satisfy the Karush-Kuhn-Tucker (KKT) conditions:

1. When the constraint is not active, the solution satisfies $\nabla f(x, y) = 0$. The Lagrangian also satisfies $\nabla \mathcal{L}(x, y, \lambda) = 0$ if and only if $\lambda = 0$.
2. When the constraint is active, the solution satisfies $g(x, y) = 0$ and $\nabla f(x, y) = -\lambda \nabla g(x, y)$ with $\lambda > 0$.

In both cases, the solution must satisfy $\lambda g(x, y) = 0$ subject to $\lambda \geq 0$ and $g(x, y) \geq 0$.

Optimisation of maximum margin classifier: minimise $\frac{1}{2}\|w\|^2$ subject to $t_n(w^\top x_n + b) \geq 1$ for all n. The Lagrangian is

$$\mathcal{L}(w, b, \boldsymbol{a}) = \frac{1}{2}\|w\|^2 - \sum_{n=1}^{N} a_n (t_n (w^\top x_n + b) - 1)$$

Differentiate with respect to w and b gives $w = \sum_{n=1}^{N} a_n t_n x_n$ and $\sum_{n=1}^{N} a_n t_n = 0$. From the KKT conditions, the solution must satisfy $a_n(t_n(w^\top x_n + b) - 1) = 0$. Hence data

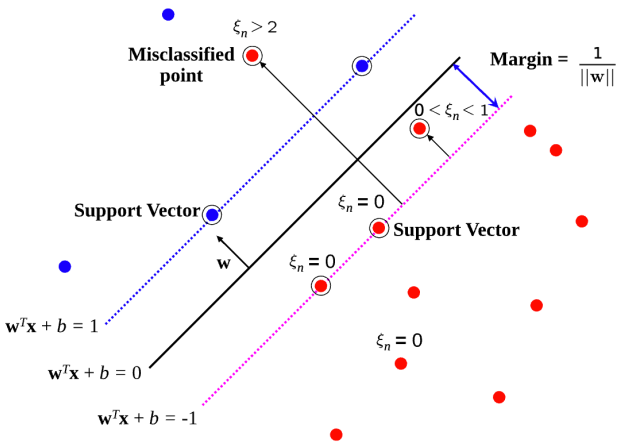
points with $a_n > 0$ are the support vectors, the predictions are determined by them:

$$y(x) = \sum_{n \in \mathcal{S}} a_n t_n x_n^\top x + b$$

where $\mathcal{S}$ is the set of indexes of support vectors. The bias can be obtrain by the constraint $t_n(w^\top x_n + b) = 1$

$$b = \frac{1}{|\mathcal{S}|} \sum_{n \in \mathcal{S}} \left(t_n - \sum_{m \in \mathcal{S}} a_m t_m x_m^\top x_n \right)$$

Soft margin: allow some data points to be misclassified by introducing slack variables $\xi_1, \ldots, \xi_N \geq 0$ in the constraints $t_n(w^\top x_n + b) \geq 1 - \xi_n$ for all n. Points with $\xi_n = 0$ are on the margin or beyond, points with $0 < \xi_n \leq 1$ are between the margin and the decision boundary, and points with $\xi_n > 1$ are misclassified.



Optimisation with soft margin: minimise $\frac{1}{2}\|w\|^2 + C\sum_{n=1}^N \xi_n$ subject to $t_n(w^\top x_n + b) \geq 1 - \xi_n$ and $\xi_n \geq 0$ for all n. The Lagrangian is

$$L(w, b, \boldsymbol{a}, \boldsymbol{\mu}) = \frac{1}{2}\|w\|^2 + C\sum_{n=1}^N \xi_n - \sum_{n=1}^N a_n(t_n(w^\top x_n + b) - 1 + \xi_n) - \sum_{n=1}^N \mu_n \xi_n$$

with Lagrange multipliers $a_n = C - \mu_n \geq 0$ and $\mu_n \geq 0$. $C > 0$ controls the trade-off between margin size and misclassification rate. Small C allows more misclassification and larger margin, while large C allows less misclassification and narrower margin. The KKT conditions are $a_n(t_n(w^\top x_n + b) - 1 + \xi_n) = 0$ and $\mu_n \xi_n = 0$. Points with $a_n = C$ lie inside the margin can either be correctly classified or misclassified.

Non-linear max-margin classifier: replace feature vectors $\boldsymbol{x}_n$ with a non-linear mapping $\boldsymbol{\phi}(\boldsymbol{x}_n) = [\phi_1(\boldsymbol{x}_n), \ldots, \phi_M(\boldsymbol{x}_n)]^\top$.

Gram matrix: the matrix of inner products of feature vectors, where $K_{nm} = \phi(\boldsymbol{x}_n)^\top \phi(\boldsymbol{x}_m)$. The kernel function $k(\boldsymbol{x}_n, \boldsymbol{x}_m) = \phi(\boldsymbol{x}_n)^\top \phi(\boldsymbol{x}_m)$ can be used to compute the Gram matrix without explicitly computing the feature vectors.

Mercer's condition: $k(\cdot, \cdot)$ is a valid kernel function if and only if 1) k is symmetric, and 2) the Gram matrix K is positive semi-definite for any set of input vectors.

Kesler's construct: add constraints to enforce output of correct classifer than the others

$$w_{t_n}^\top \phi(\boldsymbol{x}_n) + b_{t_n} - (w_j^\top \phi(\boldsymbol{x}_n) + b_j) > 0, \text{ for all } j \neq t_n$$

Merge all the weights into a single vector $\bar{w}^\top = [w_1^\top, b_1, \ldots, w_K^\top, b_K]$. The constraints can be written in the form of a binary classifier ($\ldots$ denotes zeros)

$$\bar{w}^\top \bar{\phi}(\boldsymbol{x}_n)^{(t_n j)} > 0, \text{ where } \bar{\phi}(\boldsymbol{x}_n)^{(t_n j)\top} = [\ldots, \underbrace{\phi(\boldsymbol{x}_n)^\top}_{\text{element } t_n}, 1, \ldots, \underbrace{-\phi(\boldsymbol{x}_n)^\top}_{\text{element } j}, -1, \ldots]$$

The dimensinoality of feature space if Kd and the number of constraints (effective number of data points) is $(K-1)N$.

Perceptron algorithm: Initialise $w = 0$ and $b = 0$. For each data point, if it is misclassified, update the weights and bias as $w \leftarrow w + t_n \boldsymbol{x}_n$ and $b \leftarrow b + t_n$. Repeat until all data points are correctly classified. Classfication is based on $y(\boldsymbol{x}^*) = w^\top \boldsymbol{x}^* + b$.

The kernelised version is: Initialise $\alpha_n = 0$ for all n and $b = 0$. For each data point, if it is misclassified, update the weights and bias as $\alpha_n \leftarrow \alpha_n + 1$ and $b \leftarrow b + t_n$. Repeat until all data points are correctly classified. Classification is based on $y(\boldsymbol{x}^*) = \sum_{n=1}^N \alpha_n t_n k(\boldsymbol{x}_n, \boldsymbol{x}^*) + b$.

8 Kernels for Structured Data

String kernel: Given a list of substrings $s_1, \ldots, s_m$, the feature $\phi_s(x)$ indicates the occurrence of s in x. The kernel between two strings x and x' is defined as

$$k(x, x') = \sum_s \phi_s(x)\phi_s(x')$$

Examples:

- *Gap-weighted kernel:* $\phi_s(x) = \lambda^n$ where $0 < \lambda < 1$.
- *k-spectrum kernel:* $\phi_s(x)$ is the number of occurrences of length-k sequence s in x. When $k = 1$, it becomes the bag-of-words kernel.

Random-walk graph kernel: A k-length walk in a graph is defined as a sequence of k edges. The number of k-length walks between nodes i and j is given by $[A^k]_{ij}$, where A is the adjacency matrix of the graph.

The random-walk kernel $k(G, G')$ counts the number of common walks in two graphs G and G'. Defined as $k(G, G') = [I - \lambda A \otimes A']^{-1}$ where A and A' are the adjacency matrices of G and G' respectively, $\otimes$ is the Kronecker product, and λ is a decay factor.

Weisfeiler-Lehman graph kernel: Repeat for multiple iterations, for each graph in the dataset: For each node in a graph, create a set with labels of adjacent vertices, sort the label set and add vertex label as a prefix, then compress this label sequence into a unique value. The kernel is the bag-of-words kernel on both original nodes and compressed nodes.

Fisher kernel: Train a generative model $p(x|\boldsymbol{\theta})$ and define the feature vector as the score function at maximum likelihood estimate $\phi(x_n) = \nabla_{\boldsymbol{\theta}} \log p(x_n|\boldsymbol{\theta})|_{\boldsymbol{\theta}_{\text{MLE}}}$.